L10: Deep Learning for Timeseries

CSci 560 Deep Learning w/ Python (Chollet) Ch. 10

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L10.1 Different Kinds of Timseries Tasks

CSci 560: Deep Learning w/ Python (Chollet) Ch. 10.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Timeseries Datasets

- A **timeseries** can be any data obtained via measurments at regular intervals.
- Examples include:
  - Daily price of a stock
  - Hourly electricity consumption of a city
  - Weekly sales of a store
  - Speech input sound waves caputerd at 8kHz (8,0000 sample per second)
- Working with timeseries involves understanding the **dynamics** of a system:
  - its periodic cycles
  - how it trends over time
  - its regular regime and its sudden spikes

EAST TEXAS A&M

Kinds of Timeseries Tasks

- **Forecasting**: predicting what will happen next in a timeseries.
  - The most common, for example forcast electricity consumption a few hours in advance so can anticipate demand.
- **Classification**: Assign one or more categorical labels to a timeseries.
  - Given timeseries activity of a visitor on a website, classify whether visitor is bot or human
- **Event detection**: Identify the occurrence of a specific expected event within a continuous data stream.
  - *hotword detection* to monitor a stream and detect utterences like "Hey Alexa"
- **Anomaly detection**: Detect anything unusual happenning within a continuous datastream.
  - Unusual credit card timeseries may be fraud

EAST TEXAS A&M

L10.2 A Temperature Forcasting Example

CSci 560: Deep Learning w/ Python (Chollet) Ch. 10.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

L10.3 Understanding Recurrent Neural Networks

CSci 560: Deep Learning w/ Python (Chollet) Ch. 10.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Understanding Recurrent Neural Networks

- All of the neural networks you've seen so far have no memory.
  - Each input shown is processed independently with no state kept between inputs.
  - For a dense network, we flatten five days of data into a single large vector: **feedforward networks**.
- Biological systems use memory
  - while reading you process each word one-by-one while keeping memories of what came before.
- A **recurrent neural network** (RNN) adopts the same principle.
  - Processes sequences by iterating through the sequence elements and maintaining a **state**

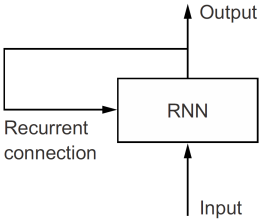


Figure 1: A recurrent network: a network with a loop. (Chollet, 2021, pg. 294)

EAST TEXAS A&M

Recurrent Operation

- A **recurrent neural network** (RNN) adopts the same principle.
  - Processes sequences by iterating through the sequence elements and maintaining a **state**
- The state of the RNN is reset between processing two different sequences, so you still consider one sequence to be a single input.
- What changes is that this data point is no longer processed in a single step
  - rather, the network internally **loops** over sequence elements

Example implementation of forward pass of a toy RNN:

```
# the state at time t
state_t = 0

# iterate over sequence elements
for input_t in input_sequence:
    output_t = f(input_t, state_t)
    # the previous output becomes the state for the next iteration
    state_t = output_t
```

EAST TEXAS A&M

Recurrent Operation

To be more specific, you can even flesh out the function `f`. It is a tensor operation on the `input_t` and the `state_t`.

```
# the state at time t
state_t = 0

# iterate over sequence elements
for input_t in input_sequence:
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    # the previous output becomes the state for the next iteration
    state_t = output_t
```

EAST TEXAS A&M

RNN Step Function

A RNN is a `for` loop that reuses quantites computed during the previous iteration of the loop.

RNNs are characterized by their step function:

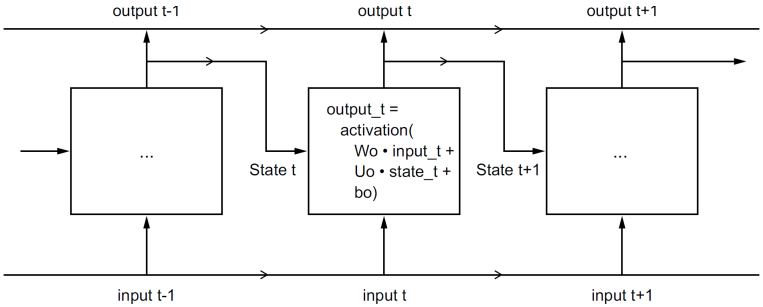


Figure 2: A simple RNN, unrolled over time. (Chollet, 2021, pg.295)

EAST TEXAS A&M

RNN Layer Output in Keras

- All recurrent layers in Keras `SimpleRNN`, `LSTM` and `GRU` can be run in two different modes
  1. they can return the full sequences of successive outputs for each time step like we did before, a rank-3 tensor of shape `(batch_size, timesteps, output_features)`
  2. or they can return only the last output for each input sequence a rank-2 tensor of shape `(batch_size, output_features)`
- These two modes are contolled by the `return_sequences` constructor arguments.

EAST TEXAS A&M

Limits of the SimpleRNN Layer

- In practice, you'll rarely work with the `SimpleRNN` layer.
- `SimpleRNN` has a major issue
  - **vanishing gradient** problem
  - same as we discussed for deep networks
  - each loop in a `SimpleRNN` acts like a layer, so if a large number of loops, gradients diminish and are hard to get signal through a simple RNN when performing wegith update.
- `LSTM` and `GRU` essentially have inner components similar to residual connections to mitigate vanishing gradients.

EAST TEXAS A&M

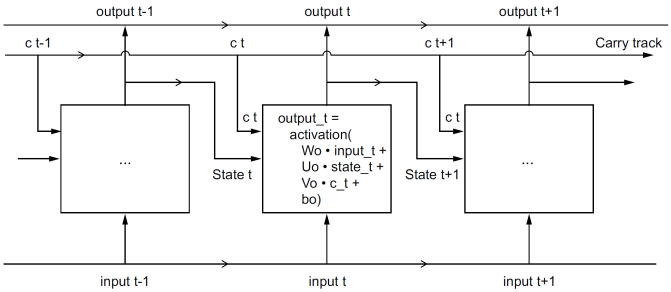
LSTM Layer: Avoiding Vanishing Gradients in Recurrent Layers

- A variant of the 'SimpleRNN
- But it adds a way to carry information across many timesteps.
- Essentially what LSTM does is to save information for later thus preventing older signals from gradually vanishing during processing.
  - e.g. similar to residual connections

EAST TEXAS A&M

Going from SimpleRNN to the LSTM

- W_o, U_o and b_o are the weights at $output_t$
- LSTM adds in additional dataflow c_t where `C` stands for **carry**.
- The carry has the following impact
  - it will be combined with the input connection and the recurrent connection followed by a bias add and the application of an activation function
  - and it will affect the state being sent to the next timestep
- Conceptually the carry dataflow is a way to modulate the next output and the next state.



Going from SimpleRNN to the LSTM: output and i,f,k activations

- The carry dataflow is computed using 3 distinct transformations.
- All three have form of a `SimpleRNN` cell transformation.
- We'll index with letters `i` `f` and `k`

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(c_t, Vo) + bo)
i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wi) + bk)
```

EAST TEXAS A&M

Going from SimpleRNN to the LSTM: compute the carry

We obtain the new carry state (the next `c_t`) by combining `i_t`, `f_t` and `k_t`

```
c_t+1 = c_t * i_t * k_t * f_t
```

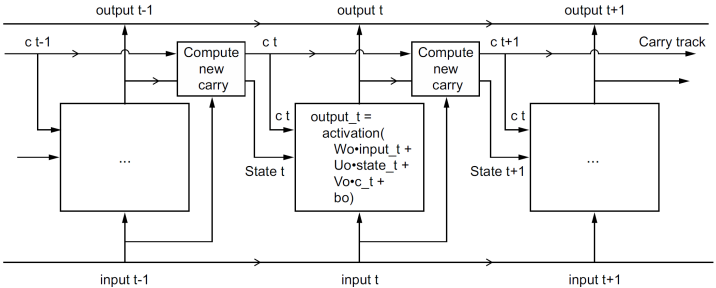


Figure 4: Anatomy of an LSTM. (Chollet, 2021, pg.299)

What these operations **actually** do is determined by the contents of the weights parameterizing them; and the weights are learned in an end-to-end fashion, starting over with each training round.

EAST TEXAS A&M

L10.4 Advanced Use of Recurrent Neural Networks

CSci 560: Deep Learning w/ Python (Chollet) Ch. 10.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

EAST TEXAS A&M

Bidirectional RNN

- A bidirectional RNN exploits the order sensitivity of RNNS
- It uses two regular RNNS
  - One see input in normal chronological order
  - The other see it in reverse chronological order
- A bidirectional RNN exploits the idea that, for some problems, reverse order is a different but useful presentation to build representations.
- Bidirectional RNNs are a gret fit for text data, or any other kind of data where order matters, yet which order you use doesn't matter.

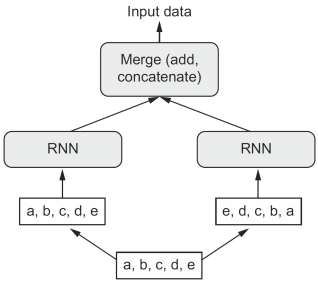


Figure 5: How a bidirectional RNN layer works. (Chollet, 2021, pg.306)

EAST TEXAS A&M

Summary of Deep Learning for Timeseries

- It's good to first establish common-sense baselines for your metric of choice. If you don't have a baseline to beat, you can't tell whether you're making real progress.
- Try simple models before expensive ones, to make sure the additional expense is justified. Sometimes a simple model will turn out to be your best option.
- When you have data where ordering matters, and in particular for timeseries data, recurrent networks are a great fit and easily outperform models that first flatten the temporal data.
- The two essential RNN layers available in Keras are the LSTM layer and the GRU layer.
- To use dropout with recurrent networks, you should use a time-constant dropout mask and recurrent dropout mask.
  - These are built into Keras recurrent layers, so all you have to do is use the recurrent_dropout arguments.
- Stacked RNNs provide more representational power than a single RNN layer. They're also much more expensive and thus not always worth it.

EAST TEXAS A&M

Bibliography

Chollet, F. (2021). *Deep learning with python* (second). Manning.

EAST TEXAS A&M